

Programming

0_hello_world

Ты кто?



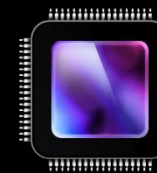
Вейлер Андрей Игоревич

✉ andrei_veiler@mail.ru

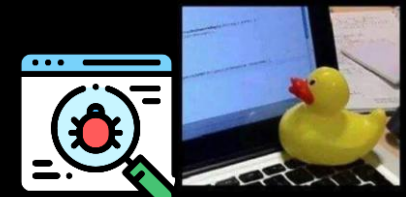
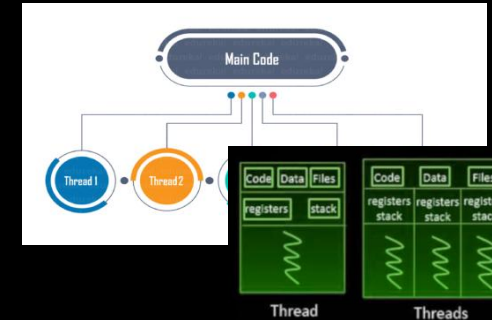
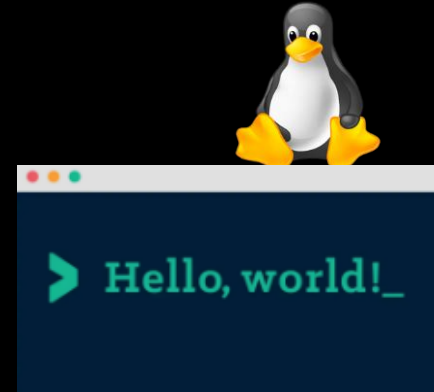
📍 @kruffka

Немного о курсе

- Разберемся с Языком С
- Подтянем технический Английский и Русский язык
- Code Style
- Узнаем как работать в консолике linux
- Система контроля версий (Git)
- Системы сборки кода
- Архитектура ПК и ПО
- Отладка программ (всякая разная)
- Базовые алгоритмы и чуть про их сложность (подробнее узнаете на САОДе)
- Жизненный цикл ПО и основы тестирования
- Основы многопоточных программ
- Насколько успеем основы C++
- Обсудим ИИ вместе с ИИ
- Ну и немного мемов (профессия разработчика нервная.. смех полезен)



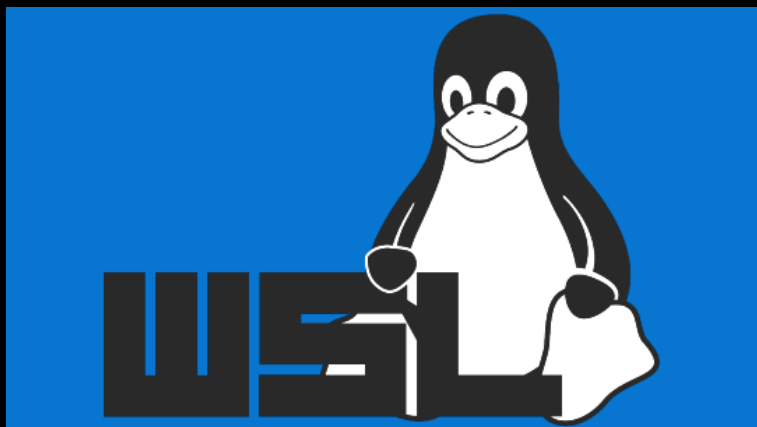
```
1011100 11010110 00001011 00011110 00111010 11110100 10101001 00011101
01011101 01010010 10000011 01000010 00011000 11101000 10011101 01111111
11101101 01100101 11000101 01010111 10001010 00001011 11100110 11010111
11001101 00111000 00001110 01010101 00110010 00111001 00110000 10111001
00000100 00010000 01100111 10100111 10111001 00000000 01101010 00110001
01101111 10110110 10000001 01000000 10100001 00001110 01001110 01001011
11011001 00000100 01000000 10011000 10010001 10111110 01001011 00010001
00000111 10010110 11101011 01101111 10011110 01101000 11001001 11110101
10000100 10000000 00010010 10111100 01000101 10110110 00000100 11111011
10111110 10110100 10011110 01111100 11100001 11110111 11100001 10001100
00010001 11000111 10111000 01110110 11101010 10001010 00011001 11010100
01011111 10110010 11000011 11110110 01101110 10110110 01010000 11000100
10001100 10101001 00010000 10011010 00011111 10101111 11001000 00100101
11000011 00111001 01111100 11010001 11011101 01110100 01100101 10110110
01110010 10011010 01000001 01100001 10010101 10110010 11101111 00100011
00101111 10011100 01010000 01011001 01110101 10101111 01001000 00110110
```



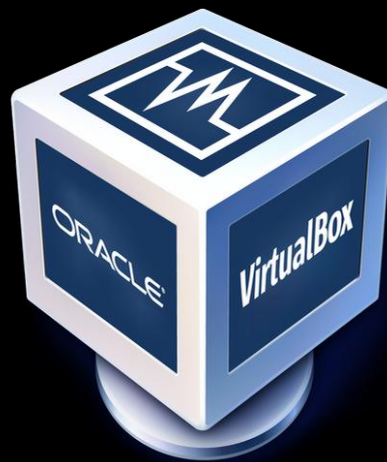
Когда и кто приходит? (практики)



Linux для ноутбуков



Windows Subsystem for Linux



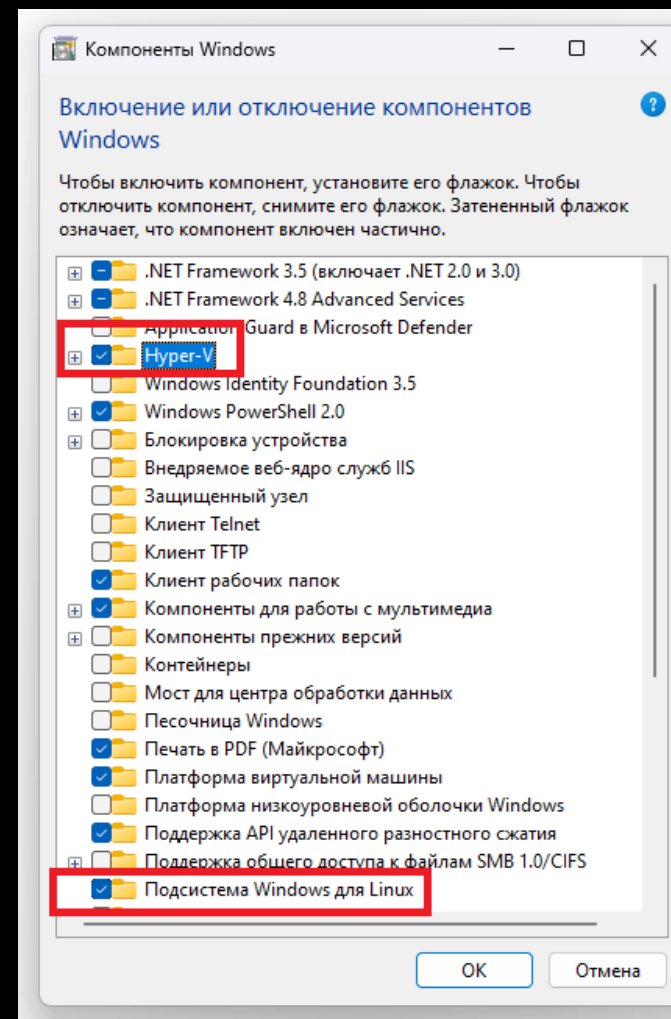
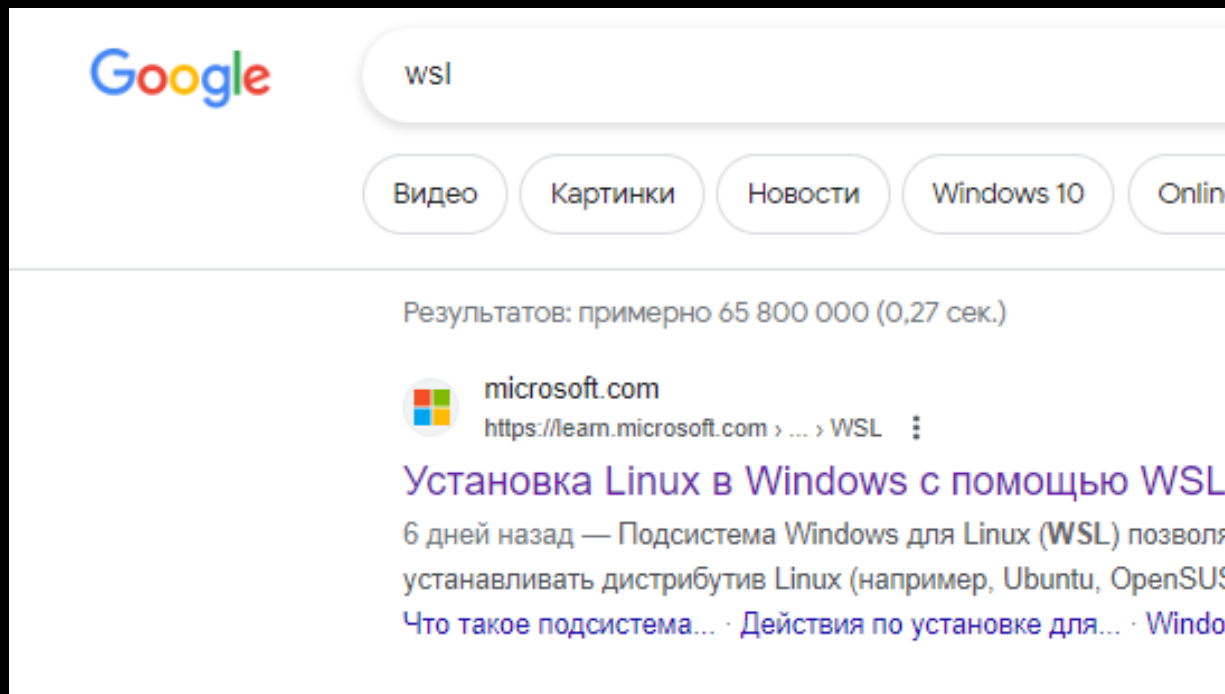
Virtual Machine + Linux



Desktop Ubuntu

*Если у вас MacOS, то ставить linux не нужно, достаточно включить режим разработчика

Windows Subsystem for Linux



Virtual Machine

Google

как установить virtualbox и ubuntu

Видео Картинки Новости Книги Покупки Карты Авиабилеты

Результатов: примерно 94 100 (0,48 сек.)

Видео

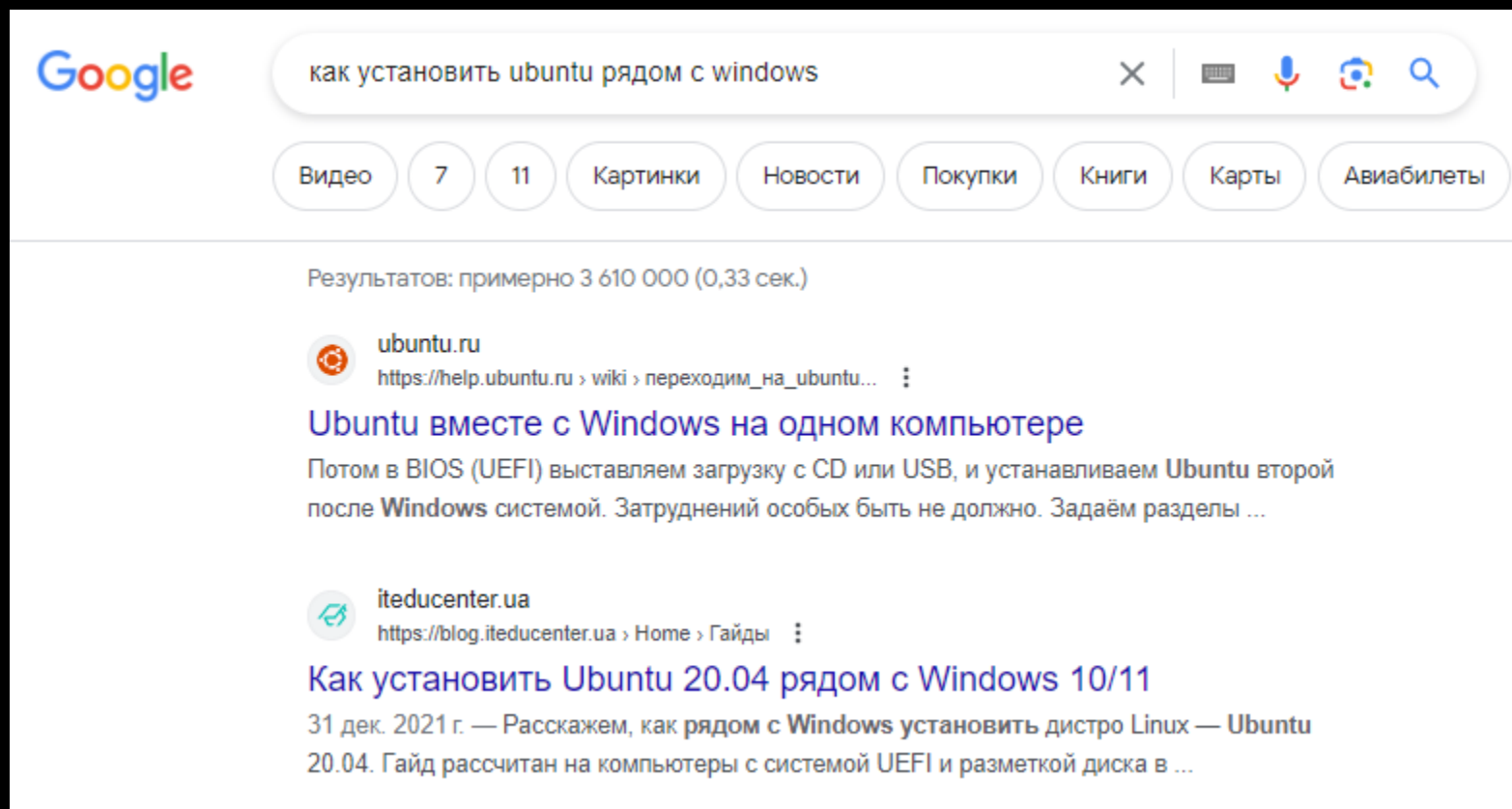
 Linux для начинающих / Урок #2 – Установка VirtualBox и ...

YouTube · Гоша Дударь
4 сент. 2021 г.

10 ключевых моментов в этом видео

				
Фрагмент c00:00 Начало	Фрагмент c00:20 Установка VirtualBox	Фрагмент c01:55 Создание виртуального окружения	Фрагмент c06:00 Настройки окружения	Фрагмент c08:32 Установочный файл Ubuntu

Ubuntu OS



The screenshot shows a Google search interface. At the top left is the Google logo. The search bar contains the text 'как установить ubuntu рядом с windows'. To the right of the search bar are icons for clearing the search, voice search, image search, and a magnifying glass. Below the search bar is a row of filters: 'Видео', '7', '11', 'Картинки', 'Новости', 'Покупки', 'Книги', 'Карты', and 'Авиабилеты'. Below the filters, the search results are displayed. The first result is from 'ubuntu.ru' with the URL 'https://help.ubuntu.ru › wiki › переходим_на_ubuntu...'. The title of the result is 'Ubuntu вместе с Windows на одном компьютере'. The description says 'Потом в BIOS (UEFI) выставляем загрузку с CD или USB, и устанавливаем Ubuntu второй после Windows системой. Затруднений особых быть не должно. Задаём разделы ...'. The second result is from 'iteducenter.ua' with the URL 'https://blog.iteducenter.ua › Home › Гайды'. The title of the result is 'Как установить Ubuntu 20.04 рядом с Windows 10/11'. The description says '31 дек. 2021 г. — Расскажем, как рядом с Windows установить дистри Linux — Ubuntu 20.04. Гайд рассчитан на компьютеры с системой UEFI и разметкой диска в ...'.

Google

как установить ubuntu рядом с windows

Видео 7 11 Картинки Новости Покупки Книги Карты Авиабилеты

Результатов: примерно 3 610 000 (0,33 сек.)

 ubuntu.ru
https://help.ubuntu.ru › wiki › переходим_на_ubuntu...
Ubuntu вместе с Windows на одном компьютере
Потом в BIOS (UEFI) выставляем загрузку с CD или USB, и устанавливаем Ubuntu второй после Windows системой. Затруднений особых быть не должно. Задаём разделы ...

 iteducenter.ua
https://blog.iteducenter.ua › Home › Гайды
Как установить Ubuntu 20.04 рядом с Windows 10/11
31 дек. 2021 г. — Расскажем, как рядом с Windows установить дистри Linux — Ubuntu 20.04. Гайд рассчитан на компьютеры с системой UEFI и разметкой диска в ...

Eios



Кодовое слово:
ИКС

<https://eios.sibsutis.ru/course/view.php?id=3912>

Github



<https://github.com/kruffka/C-Programming>

Домашка: зарегистрироваться на github.com. Желательно использовать gmail, т.к. с другими могут возникнуть проблемы при регистрации

Программирование

Программа - это просто набор инструкций, которые кто-то (или что-то) выполняет для получения некоторого результата.

Классический пример - рецепт, пусть будет рецепт мильфея (хрустящий десерт с малиной) записанный на бумаге. Сам рецепт написан на бумаге на языке **программирования**, который понимает исполнитель. Исполнителем в этом хаосе пусть будет некий Повар, который следует четко по инструкции для приготовления мильфея. И вот сам процесс написания рецепта для повара и есть **программирование**.

Если вернуться в реальный мир, то найдем программы повсюду:

Телефоны, ПК, умные чайники, стиралки, самолеты, машины, мессенджеры, браузеры, сайт, ИИ для генерации котиков..



Язык программирования

Язык — это заданный набор символов и правил, устанавливающих способы комбинации этих символов между собой для записи осмысленных текстов.

Язык программирования (ЯП) — это формальный, человеко-машинный язык для записи компьютерных программ, который состоит из набора правил (**лексических, синтаксических и семантических**) для создания инструкций, понятных компьютеру.



Лексика, Семантика и Синтаксис

Лексика (Lexicon) - относится к набору допустимых символов и токенов в языке программирования.

Отвечает на вопрос: Какие слова и символы можно использовать в предложении?..

Ключевые слова типа "for", "if", "else", "while"..

Идентификаторы - допустимые имена переменных.

Операторы - +, -, &&, |, ~ и т.д.

Разделители - символы, разделяющие токены, пример " ", ":", " ", "..."

Семантика (Semantics) - относится к значению и смыслу программных конструкций.

Отвечает на вопрос: Является ли написанное предложение корректным? Если да, то что оно делает?..

Как интерпретировать выражения, пример "x++" - прибавить к значению "x" единицу и записать результат в "x"

Синтаксис (Syntax) - относится к правилам и структуре языка программирования.

Отвечает на вопрос: Как построить правильное предложение?..

Структура предложения: Как правильно написать цикл "while" или условие "if-else", куда ставить круглые скобки, а куда точку с запятой

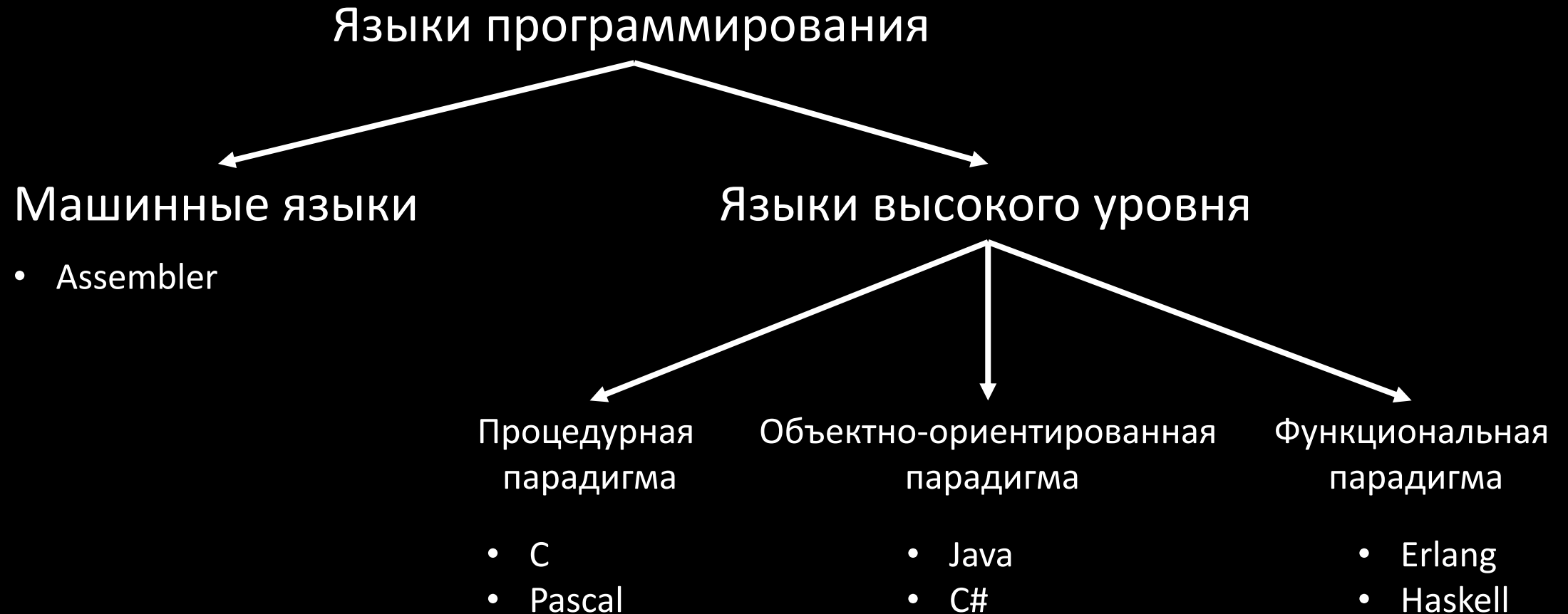
Эти правила обычно описаны в стандарте ЯП

Немного истории

- Машины в прошлом веке понимали только 0 и 1 (сейчас тоже так)
- В 50-х годах придумали ассемблер и мнемоники
- Спустя годы появились первые языки Высокого Уровня (Fortran, LISP и т.д.)
- Язык Си появился в 70-х годах. Создатель Деннис Ритчи

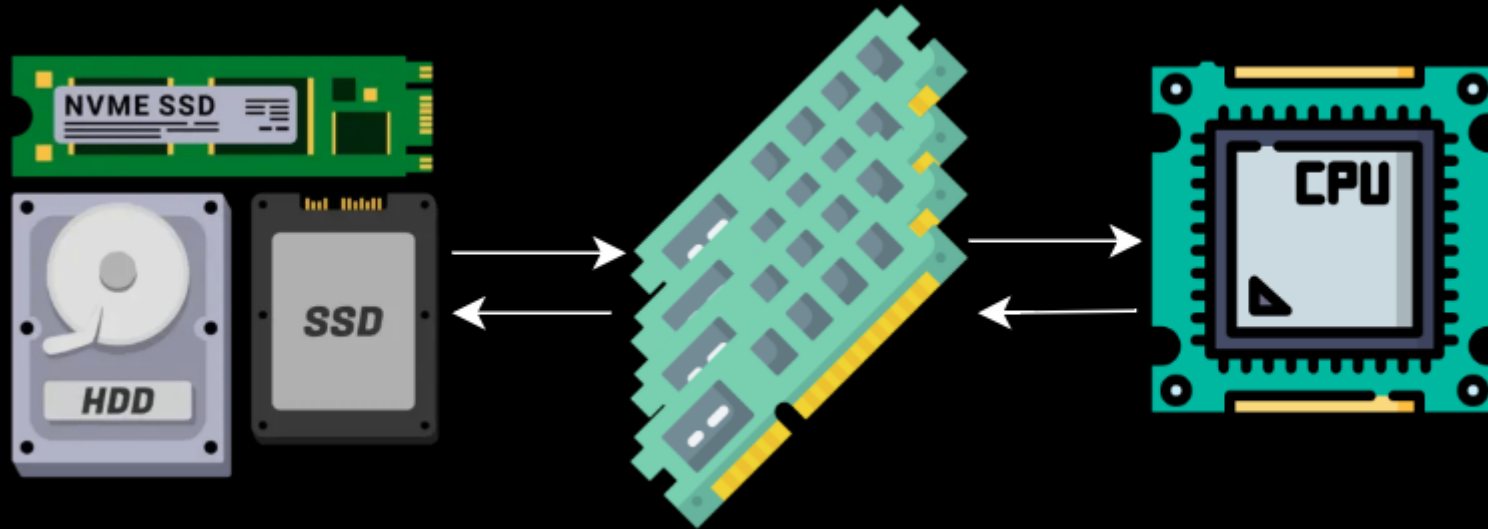


Классификация языков программирования



Каждый ЯП и парадигма созданы для решения своих задач

Machine Code



```
01001000 01100101 01101100 01101100 01101111 00101100  
00100000 01110111 01101111 01110010 01101100 01100100  
01001000 01100101 01101100 01101100 01101111 00101100  
00100000 01110111 01101111 01110010 01101100 01100100
```

Source code

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("hello, world!\n");
```

```
}
```

Трансляция

Компиляция/Интерпретация

- 1) Лексический анализ
- 2) Синтаксический анализ
- 3) Семантический анализ
- 4) Оптимизация
- 5) Генерация кода

- 1) Прочитать инструкцию
- 2) Определить соответствующие действия
- 3) Выполнить соответствующие действия
- 4) Если не конец программы,
то выполнить п.1



C language

Стандарт языка СИ

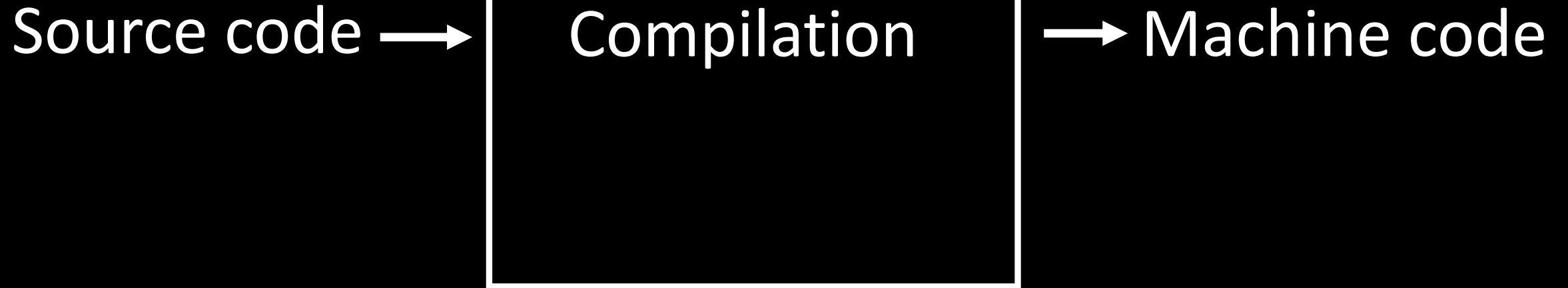


Реализация стандарта языка СИ

GCC

Clang

Compilation



Hello, C

```
#include <stdio.h>

int main()
{
    printf("hello, world!\n");
    return 0;
}
```

Традиция у программистов на любом языке первая программа — это вывод на экран текста "hello, world!".

Hello, errors

Пробуем `print` вместо `printf`

Hello, errors

Пробуем include вместо #include

Variables

Представим себе **коробку**, в которую можно класть яблоки.

Возьмем маркер и напишем на коробке - "коробка яблок". Можно взять эту "коробка яблок" и накидать в нее яблок (записать что-то в переменную), а потом взять и посмотреть сколько там сейчас всего яблок (прочитать из переменной).

При этом коробка **не бесконечная**, в нее влезет определенное количество яблок. Можно создать коробку с гречкой, в нее яблоки складывать не будем, т.к. эти коробки предназначены для разных типов продуктов



Data types

Тип	Формат	Размер (байт)	Диапазон знаковый (signed)	Диапазон беззнаковый (unsigned)
char	%c, %hhhd	1	-128 до 127	0 до 255
short int short	%hd	2	-32 768 до 32 767	0 до 65 535
int	%d	4	-2 147 483 648 до 2 147 483 647	0 до 4 294 967 295
long	%ld	8	-9 223 372 036 854 775 808 до 9 223 372 036 854 775 807	0 до 18 446 744 073 709 551 615
long long	%lld	8	-9 223 372 036 854 775 808 до 9 223 372 036 854 775 807	0 до 18 446 744 073 709 551 615
float	%f	4	$\pm 1.2 \times 10^{-38}$ до $\pm 3.4 \times 10^{38}$	-
double	%lf	8	$\pm 2.3 \times 10^{-308}$ до $\pm 1.7 \times 10^{308}$	-
long double	%Lf	16	$\pm 3.4 \times 10^{-4932}$ до $\pm 1.1 \times 10^{4932}$	-

*Формат - подсказка какой **специальный символ** использовать для вывода через **printf**

Data types

- **char** - сокращение от character (символ), используется для символов (в целых числах)
- **int** - сокращение от integer (целое), тип данных для хранения целых
- **float** - что-то плавающее с англ. (значение с плавающей точкой), вещественные числа
- **double** - как float, но с двойной точностью

Все типы данных, хранящие **ЦЕЛЫЕ** числа могут быть как знаковыми (**signed**) так и беззнаковыми (**unsigned**)

Variable declaration

```
int a = 5;
```

- **int** - тип данных **целые числа**
- **a** - имя переменной
- **=** - оператор присвоения
- **5** - целая константа
- **;** - конец инструкции

Объявляем переменную 'a' типа `int` равной 5

Variable declaration

```
int b;  
b = -5;
```

В языке Си при объявлении переменной без присваивания значения **int b;** - не дает гарантий, что **b** по умолчанию будет равна **0**. Значение может быть равно некоторому **мусору** - большому случайному числу.

Constant

Константа - значение, что нельзя изменить. Константы бывают разных типов данных и обычно их мы кладем в переменные.

- 123 - int, просто целое число
- 123u - unsigned int, буковка 'u' или 'U' в конце
- 123L - long, буква 'l' или 'L'
- 123LU - unsigned long int, 'lu' или 'LU'
- 5.0 - double, стоит точка
- 42.1f - float, есть буква 'f' или 'F'
- 122.88L - long double, 'Lf' или точка и 'L'
- '#' - char, символ решетки, все символы заключаются в 'одинарные' кавычки

Constant

```
int a = 5;  
char b = 'A';  
short c = -12;  
float d = 5.132;  
double e = 5.0;
```

Format

- %d - **d**ecimal, целое со знаком в десятичной системе счисления
- %u - **u**nsigned decimal, целое беззнаковое
- %c - **c**har, символьное
- %hd - **h**short **d**ecimal, короткое десятичное (hhd = short short **d**ec, hhu = short short **u**nsigned)
- %f - **f**loat, вещественное

Var names



Прошлогодний мем по стилю кода

Var names

Кринж

```
int b = 420;  
double c = 299792458;
```

База

```
int toyPrice = 420;  
double speed_of_light = 299792458;
```

Looong looong name

Умейте вовремя остановиться



Overflow

```
unsigned int money = 2147483648;  
printf("double of money is %u$\n", money * 2);
```

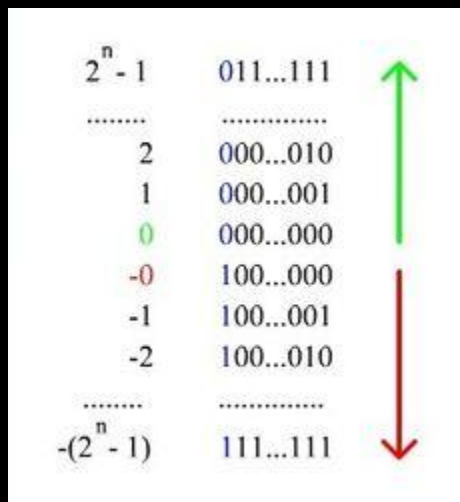
Negative numbers

Прямой код



5 = 00000101

-5 = 10000101



+ Прост в вычислении

+ Количество «+» чисел равно количеству «-» чисел

- Два нуля (-0 и +0)

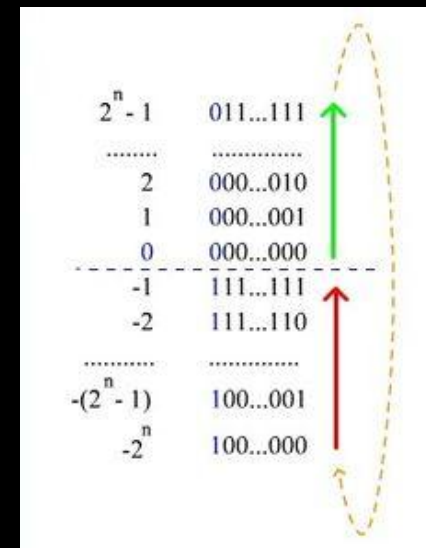
- Требуется реализации операции вычитания

Дополнительный код (дополнение до двух)



5 = 00000101

-5 ->



1. Прямой код 00000101
2. Обратный код 11111010
3. Прибавляем 1 11111011
4. Единица в s 11111011

Negative numbers

Самый старший бит в знаковой переменной отвечает за знак! '1' - число отрицательное, '0' - число положительное. Если переменная unsigned, то старший бит не используется для знака, отсюда допустимый диапазон больше на 1 бит.

Например для char мы получаем не -128..127, а 0..255



Какой диапазон чисел будет для signed int?

ASCII

Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex	
0	00	NUL	16	10	DLE	32	20		48	30	@	64	40	P	80	50	p	96	60	`
1	01	SOH	17	11	DC1	33	21	!	49	31	A	65	41	Q	81	51	q	97	61	a
2	02	STX	18	12	DC2	34	22	"	50	32	B	66	42	R	82	52	r	98	62	b
3	03	ETX	19	13	DC3	35	23	#	51	33	C	67	43	S	83	53	s	99	63	c
4	04	EOT	20	14	DC4	36	24	\$	52	34	D	68	44	T	84	54	t	100	64	d
5	05	ENQ	21	15	NAK	37	25	%	53	35	E	69	45	U	85	55	u	101	65	e
6	06	ACK	22	16	SYN	38	26	&	54	36	F	70	46	V	86	56	v	102	66	f
7	07	BEL	23	17	ETB	39	27	'	55	37	G	71	47	W	87	57	w	103	67	g
8	08	BS	24	18	CAN	40	28	(	56	38	H	72	48	X	88	58	x	104	68	h
9	09	HT	25	19	EM	41	29	)	57	39	I	73	49	Y	89	59	y	105	69	i
10	0A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	Z	90	5A	z	106	6A	j
11	0B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	[	91	5B	{	107	6B	k
12	0C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	\	92	5C		108	6C	l
13	0D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	]	93	5D	}	109	6D	m
14	0E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	~	110	6E	n
15	0F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	DEL	111	6F	o

IEEE-754

$$(-1^s) * m * b^e$$

s – знак (0 положительное число, 1 отрицательное)

m – мантисса

b – основание

e – экспонента

$$122.88 = (-1^0) * 12288 * 10^{-2}$$

$$s = 0$$

$$m = 12288$$

$$b = 10$$

$$e = -2$$



$$0.2 + 0.1 \neq 0.3$$

<https://0.30000000000000004.com/>

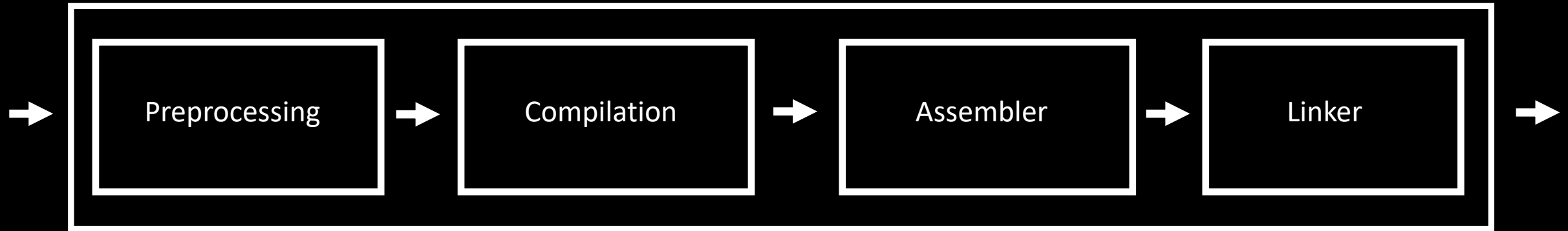
Operators

Операция (выражение)		Оператор	Синтаксис выражения
Присваивание		=	a = b
Сложение		+	a + b
Вычитание		-	a – b
Унарный плюс		+	+a
Унарный минус		-	-a
Умножение		*	a * b
Деление		/	a / b
Операция модуль (остаток от деления целых чисел)		%	a % b
Инкремент	префиксный	++	++a
	суффиксный	++	a++
Декремент	префиксный	--	--a
	суффиксный	--	a--

Assignment

Операция (выражение)	Оператор	Синтаксис выражения	Значение
Сложение	+	$a += b$	$a = a + b$
Вычитание	-	$a -= b$	$a = a - b$
Умножение	*	$a *= b$	$a = a * b$
Деление	/	$a /= b$	$a = a / b$
Операция модуль (остаток от деления целых чисел)	%	$a \% = b$	$a = a \% b$

Compilation steps



- 1) Обработка препроцессором
- 2) Трансляция программы из языка СИ в Ассемблер
- 3) Трансляция из Ассемблера в объектный модуль
- 4) Объектный модуль в исполняемую программу

- 1) `gcc -E hello.c -o hello.i`
- 2) `gcc -S hello.i -o hello.s`
- 3) `gcc -c hello.s -o hello.o`
- 4) `gcc hello.o -o hello`

Список литературы

- **«Язык программирования Си»** (англ. The C Programming Language) — книга Брайана Кернигана и Денниса Ритчи
- [Стандарт языка Си](https://www.iso-9899.info/wiki/The_Standard)
- Гарвардский курс CS50 (Computer Science) любого года абсолютно бесплатно в ютубе каждый год, есть даже на русском
- Курсы молодого бойца Си от МФТИ

List of commands

Команды в терминале

```
mkdir my_dir # Создать каталог my_dir
ls           # Просмотреть список файлов в текущем каталоге
pwd          # Вывести имя текущей директории
cd my_dir    # Перейти в каталог my_dir
cd ..        # Подняться на каталог "выше" (вернуться назад)
nano hello.c # открыть файл в текст. редакторе nano
rm file_name # удалить файл с именем file_name
man command  # Мануал (Описание команды)
```

Команды в текст. редакторе nano

```
CTRL+O (^O) # save program
CTRL+X (^X) # exit program
```

Компиляция и запуск

```
gcc hello.c -o hello
./hello
```